Algorithms Mastery

A self-paced course for algorithmic reasoning and implementation

Primary path: Reason, implement, and solve unfamiliar problems

Language practice: Go first | Selected C++ and Rust practice

Course philosophy: *Model. Derive. Implement. Test. Explain.*

Advance by demonstrated skill, without stage deadlines.

Contents

Mission and target learner 3

How the curriculum works 3

Curriculum map 4

Stage 0 - Baseline and setup 5

Stage 1 - Reasoning, complexity and correctness 6

Stage 2 - Arrays, hashing and prefix information 6

Stage 3 - Two pointers and sliding windows 7

Stage 4 - Binary search and divide and conquer 7

Stage 5 - Stacks, deques, heaps and amortization 8

Stage 6 - Trees and recursive summaries 8

Stage 7 - Graph modeling, BFS and DFS 9

Stage 8 - Weighted paths and graph optimization 9

Stage 9 - Greedy reasoning, intervals and sweeps 10

Stage 10 - Backtracking and exponential search 10

Stage 11 - Dynamic programming: deriving states 11

Stage 12 - Dynamic programming: structural variants 11

Stage 13 - Strings, tries and pattern matching 12

Stage 14 - Range queries and ordered information 12

Stage 15 - Advanced graphs and reductions 13

Stage 16 - Counting, number theory and geometry 13

Stage 17 - Composition and the expanding frontier 14

Stage gates and mastery criteria 14

The 45-minute Hard benchmark 15

Language practice: Go, C++ and Rust 16

Session planning 17

Recommended resource stack 19

Selected references 19

Mission and target learner

Target: solve unfamiliar LeetCode Hard problems correctly in 45 minutes or less. Use Go first, with selected C++ and Rust practice. Choose your session duration. Advance by skill, without stage deadlines. See Language practice and Session planning.

> *"Any Hard" is the long-term ambition. The course measures progress on broad, unseen samples; no finite benchmark can certify every problem.*

How the curriculum works

Tell the tutor your available study minutes before each session. The tutor asks if you omit the duration. Split ordinary study into retrieval, focused work, and feedback using Session planning. Timed assessments keep their stated limits and a separate review period.

Day	Main work
Monday	Study one small idea and explain a worked example.
Tuesday	Close the resource; reconstruct, implement and test.
Wednesday	Solve a related problem with help only when needed.
Thursday	Change a constraint or solve another application.
Friday	Revisit older material and attempt mixed, unlabeled problems.

The problem-solving loop

- **Model:** restate constraints, try tiny cases and write a brute-force baseline.
- **Derive:** identify useful structure; define state and why each step is safe.
- **Implement:** write the solution yourself.
- **Verify:** test boundaries and compare with a simple independent solver where feasible.
- **Explain:** give the correctness argument, time/space cost and key assumptions.

Teaching and feedback

Each lesson targets an independent result with success criteria and a stopping point. Prepare demonstrations, available support, and a different application.

In a requested or agreed professor block, the instructor demonstrates how to choose an approach and supporting claims. The block has an explicit endpoint. The tutor role resumes after the example.

The tutor role is the default during your attempts. The tutor gives the task, constraints, and target. You choose the approach, claims, and argument. Questions receive narrow answers without the next solution step.

Supported practice means help is available after your attempt or request. Struggle and silence do not authorize hints. The tutor offers help before revealing it.

Requested feedback identifies the first substantive error. You make the repair. After two unsuccessful prompts, the tutor offers a hint, another example, or a teaching block and waits for your choice. This limit is a teaching safeguard, not a research threshold.

You write the reasoning and implementation. Use tests, proofs, and reliable references to make sure that claims are correct. Record assistance separately. Timed assessments remain unaided, and generated tasks need a known correct solution before assessment.

After each major block, send code, reasoning and test evidence to an experienced engineer or tutor for asynchronous review. Ask them to check the proof, complexity and boundary cases.

Make it stick

Return after	Retrieve without notes
About 2 days	State the invariant or recurrence; trace a new example.
About 1 week	Reimplement the idea or solve a related problem.
About 3 weeks	Solve a new application without its topic label.
About 6 weeks	Include it in mixed, timed practice.

Keep spaced retrieval near 15-25 percent of study time. Select useful reviews within that budget. A due date does not require the original exercise. Use short, varied prompts and bounded repairs.

Retire an overused prompt while preserving its history. Retirement does not establish mastery of the skill. Do not make one familiar proof the default review or let targeted reviews indefinitely replace fresh problem-solving.

Judge progress through whole arguments, working code, meaningful tests, and correct cost analysis. Prompted agreement and repeated paraphrases do not establish independent construction. After two teaching sessions on a repair, compare independent output and help required. If progress stalls, change the instruction or task scope.

The learning loop uses worked examples, retrieval, spacing, interleaving and feedback. Supporting research and its limits are in Selected references.

Curriculum map

Stage	Focus	Page
0	Baseline and setup	5
1	Reasoning, complexity and correctness	6
2	Arrays, hashing and prefix information	6
3	Two pointers and sliding windows	7
4	Binary search and divide and conquer	7
5	Stacks, deques, heaps and amortization	8
6	Trees and recursive summaries	8
7	Graph modeling, BFS and DFS	9
8	Weighted paths and graph optimization	9
9	Greedy reasoning, intervals and sweeps	10
10	Backtracking and exponential search	10
11	Dynamic programming: deriving states	11
12	Dynamic programming: structural variants	11
13	Strings, tries and pattern matching	12
14	Range queries and ordered information	12
15	Advanced graph structure and reductions	13
16	Counting, number theory and geometry	13
17	Composition and the expanding frontier	14

Stages 1-7 build foundations; 8-12 develop algorithm selection; 13-16 broaden coverage. Stage 17 combines techniques and remains open for new gaps.

Practice links use E / M / H for LeetCode difficulty. CSES problems are labeled separately. Named exercises are training material; keep unseen problems for assessment.

Stage 0 - Baseline and setup

Objective: identify whether your main gaps are mathematical reasoning, algorithm selection, implementation or speed.

Set up once

Start in a Go workspace with tests and a scratch file. Review slices/aliasing, maps, sorting comparators, heaps, queue indices, bytes/runes, recursion depth and integer bounds. Build core structures once, then use permitted library tools.

For selected exercises, compare the efficient algorithm against a simple brute-force solver on small generated inputs. Go fuzzing is optional. Tests complement the correctness argument.

Baseline tasks

1. Attempt selected MIT 6.006 Problem Set 0 questions before reading solutions. Separate unfamiliar Python/notation from conceptual gaps; translate programming tasks into Go.
2. Analyze three loop/recursion snippets and explain why binary search terminates.
3. Implement a queue and graph traversal if you already know them.
4. Attempt one unfamiliar Easy and one unfamiliar Medium problem.

Choose the next action

Observed gap	Response
Syntax or standard-library friction	Use drills in the exercise language before adding a new algorithm.
Cannot justify correctness	Start Stage 1; practice invariants and induction.
Understands solutions but cannot choose	Practice modeling and mixed, unlabeled tasks.
Finds the idea but code fails	Build smaller pieces; test boundaries and compare with brute force.
Correct but slow	Separate modeling, coding and debugging time; work on the bottleneck.

Start the first learning cycle

Read a short explanation of your weakest foundational idea. Trace one example, close the material, implement it and solve a related problem. Return after a delay and reconstruct it without notes.

Use one explanation source at a time. MIT supplies foundations; The Algorithm Design Manual (ADM) is a practical reference. Use the advanced sources only for the stage you are studying.

Progression rules

Complete the build task, practice ladder and exit criteria for each stage, then use the shared gate in Stage gates and mastery criteria. Split broad stages into smaller units. Skip familiar material only after an independent check.

A Hard exercise early in the course is a learning target. Learn to derive and implement it before demanding a 45-minute solve. Keep sessions bounded; resume tomorrow when necessary.

Keep the baseline in Go. Add C++ and Rust through selected practice, as described in Language practice.

Stage 1 - Reasoning, complexity and correctness

Prerequisites: Stage 0.

Objective: Explain correctness and estimate whether an algorithm fits the constraints.

Core topics: Asymptotic bounds; worst-case and amortized cost; sums/recurrences; invariants; induction; recursion; input size versus numeric value.

Recommended study: MIT: Introduction. ADM: algorithm design and analysis.

Build task: Implement linear search and merge sort. Prove each correct; analyze five loop/recursion snippets and compare predicted growth with measurements.

Practice ladder

- Two Sum (E).
- Valid Parentheses (E).

Variation: Explain why brackets need a stack; give a counterexample to counting alone.

Exit gate: Correct code, valid invariant/induction arguments, and correct time/auxiliary-space bounds. Measurements support analysis; they do not prove it.

Stage 2 - Arrays, hashing and prefix information

Prerequisites: Stage 1.

Objective: Replace repeated work with useful stored information.

Core topics: Array/slice costs; sets; frequency maps; prefix sums; in-place placement; expected hash-table cost; coordinates versus values.

Recommended study: MIT: Data Structures and Hashing. ADM: data structures.

Build task: Implement prefix sums and a frequency-based subarray counter. Compare with brute force on small inputs, including zeros, duplicates and negatives.

Practice ladder

- Group Anagrams (M).
- Subarray Sum Equals K (M).
- First Missing Positive (H).

Variation: Change existence to counting, then to recovering indices. Decide whether the same state suffices.

Exit gate: Derive map keys/values from the query; justify update order, duplicate handling and space cost.

Stage 3 - Two pointers and sliding windows

Prerequisites: Stage 2.

Objective: Move pointers only when the discarded possibilities cannot improve the answer.

Core topics: Sorted scans; fixed/variable windows; validity predicates; frequency bookkeeping; signed-data limitations.

Recommended study: Handbook: two pointers. ADM: sorting/searching as needed.

Build task: Implement longest-valid and minimum-cover windows. Test repeated requirements and empty cases; explain why each element is processed O(1) times.

Practice ladder

- Longest Substring Without Repeating Characters (M).
- Minimum Window Substring (H).

Variation: Replace uniqueness with a repetition cap. Derive exactly-k counts from at-most counts.

Exit gate: State a valid invariant and justify every endpoint move. Explain when negatives invalidate a sum-window argument.

Stage 4 - Binary search and divide and conquer

Prerequisites: Stages 1-3.

Objective: Prove which part of a search space can be discarded.

Core topics: Half-open intervals; lower/upper bounds; monotone predicates; answer-space search; merge sort; partitioning; recurrence analysis.

Recommended study: MIT: Sorting. Handbook: sorting and binary search.

Build task: Implement lower_bound and upper_bound from memory. Test empty, singleton, duplicate and all-equal inputs; separately prove an answer-search feasibility check.

Practice ladder

- Binary Search (E).
- Search in Rotated Sorted Array (M).
- Split Array Largest Sum (H).
- Median of Two Sorted Arrays (H, stretch).

Variation: Search for the first feasible capacity; identify a change that breaks monotonicity.

Exit gate: Prove termination and interval preservation. Include feasibility-check work in the complexity bound. The median problem is a later stretch.

Stage 5 - Stacks, deques, heaps and amortization

Prerequisites: Stages 2-4.

Objective: Maintain useful candidates and discard those that cannot matter again.

Core topics: Monotonic stacks/deques; dominance; stale indices; heaps; streaming selection; two-heap invariants; amortized analysis.

Recommended study: MIT: Binary Heaps. Handbook: amortized analysis. ADM: priority queues.

Build task: Build a binary heap, then use the exercise language's library heap (Language practice). Implement a monotonic deque and justify each removal.

Practice ladder

- Daily Temperatures (M).
- Largest Rectangle in Histogram (H).
- Sliding Window Maximum (H).
- Merge k Sorted Lists (H).
- Find Median from Data Stream (H).

Variation: Solve Shortest Subarray with Sum at Least K (H); explain why a deque handles signed data.

Exit gate: Prove O(n) aggregate deque work and preserve heap order/balance. Count total pushes/pops, not loop nesting.

Stage 6 - Trees and recursive summaries

Prerequisites: Stages 1, 5.

Objective: Return exactly the information a subtree's parent needs.

Core topics: Traversals; explicit stacks; BST ordering; ancestors; serialization; subtree DP; local returns versus global optima.

Recommended study: MIT: Binary Trees. ADM: trees; study AVL invariants conceptually.

Build task: Write recursive/iterative traversals and state each recursive contract. Build a serializer/deserializer with round-trip tests.

Practice ladder

- Validate Binary Search Tree (M).
- Lowest Common Ancestor of a Binary Tree (M).
- Binary Tree Maximum Path Sum (H).
- Serialize and Deserialize Binary Tree (H).

Variation: Sketch the extra parent-to-child information needed to answer for every root; implement rerooting in Stage 12.

Exit gate: Handle skewed and all-negative trees. Distinguish an extendable path from an optimum completed inside a subtree.

Stage 7 - Graph modeling, BFS and DFS

Prerequisites: Stages 5-6.

Objective: Find the graph hidden inside a problem.

Core topics: Adjacency lists; directed/undirected edges; visited states; BFS layers; DFS colors; components; cycles; topological order; implicit graphs.

Recommended study: MIT: BFS and DFS. ADM: graph traversal.

Build task: Implement BFS with path recovery and DFS/topological sorting. Model a grid and a dependency workflow; test cycles and unreachable/isolated vertices.

Practice ladder

- Number of Islands (M).
- Course Schedule II (M).
- Word Ladder (H).

Variation: Add parity, a resource budget or collected keys. Decide whether visited[vertex] still works.

Exit gate: Choose sufficient state, bound V and E including generated neighbors, recover paths and detect cycles.

Stage 8 - Weighted paths and graph optimization

Prerequisites: Stages 5, 7.

Objective: Choose a shortest-path algorithm from the edge and state constraints.

Core topics: Relaxation; Dijkstra; stale heap entries; Bellman-Ford; DAG paths; 0-1 BFS; negative cycles; expanded states; MST versus shortest paths.

Recommended study: MIT: Weighted Shortest Paths, Bellman-Ford, Dijkstra. ADM: weighted graphs.

Build task: Implement Dijkstra with path recovery and 0-1 BFS. Check Dijkstra against Bellman-Ford on random nonnegative graphs.

Practice ladder

- Network Delay Time (M).
- Cheapest Flights Within K Stops (M).
- Minimum Cost to Make at Least One Valid Path in a Grid (H).

Variation: Add a stop budget and a limited-use discount; explain why one cost per vertex may be insufficient.

Exit gate: State each algorithm’s assumptions, handle stale work, justify state and bound its expanded cost.

Stage 9 - Greedy reasoning, intervals and sweeps

Prerequisites: Stages 4-5.

Objective: Prove a local choice preserves an optimal answer.

Core topics: Exchange/staying-ahead arguments; interval ordering; selection versus scheduling; sweeps; boundary conventions; tie handling.

Recommended study: Erickson: Greedy Algorithms. ADM: greedy design and sorting.

Build task: Implement interval scheduling and an event sweep. Check small cases by enumerating subsets; find counterexamples to alternative sort orders.

Practice ladder

- Non-overlapping Intervals (M).
- Gas Station (M).
- Candy (H).

Variation: Add interval weights. Explain why the greedy proof fails and derive a DP after Stage 11.

Exit gate: Give a valid argument for the actual decision and a counterexample to an invalid alternative.

Stage 10 - Backtracking and exponential search

Prerequisites: Stages 1, 6-7.

Objective: Search completely without duplicate work or unsafe pruning.

Core topics: Choice/state definitions; recursion trees; rollback; symmetry; bitmasks; branch ordering; pruning; output-sensitive complexity.

Recommended study: ADM: combinatorial search. Erickson: Backtracking. Handbook: bit manipulation.

Build task: Implement subsets, permutations and a constraint solver. Compare pruned search with exhaustive enumeration on small inputs.

Practice ladder

- Subsets (M).
- Combination Sum (M).
- N-Queens (H).
- Sudoku Solver (H).

Variation: Add duplicates or ask only for counts; justify which symmetry rules preserve every distinct answer.

Exit gate: Enumerate answers exactly once, restore state, justify every prune and bound work by branching, depth and output.

Stage 11 - Dynamic programming: deriving states

Prerequisites: Stages 1, 7, 10.

Objective: Derive reusable subproblems and their dependencies.

Core topics: State; recurrence; base cases; evaluation order; memoization; optimal substructure; overlapping work; reconstruction; pseudo-polynomial cost.

Recommended study: MIT: Dynamic Programming. Erickson: introductory DP examples.

Build task: Write state, recurrence, bases, order and cost before coding. Implement memoized and bottom-up versions; compare with brute force.

Practice ladder

- House Robber (M).
- Coin Change (M).
- Edit Distance (M).
- Distinct Subsequences (H).

Variation: Change optimization to counting or reconstruction; check base values and whether order creates distinct outcomes.

Exit gate: Prove state preserves all relevant history and derive the recurrence independently. Check numeric capacities in bounds such as O(nW).

Stage 12 - Dynamic programming: structural variants

Prerequisites: Stages 6, 8-11.

Objective: Choose a decomposition when a simple prefix state is insufficient.

Core topics: Interval, tree/rerooting, DAG, subsequence, subset/bitmask and digit DP; state compression; safe space optimization.

Recommended study: Erickson: Dynamic Programming. Handbook: DP, trees and bit techniques.

Build task: Build interval, subset and tree DPs. Derive interval DP from the last decision; count states times transitions and check small cases exhaustively.

Practice ladder

- Longest Increasing Subsequence (M).
- Burst Balloons (H).
- Regular Expression Matching (H).
- Elevator Rides (CSES).

Variation: Compare O(n^2) LIS with a faster frontier; explain its meaning. Implement the rerooting sketched in Stage 6.

Exit gate: Derive three different DP structures, including interval and subset/tree. Prove dependencies before compressing storage.

Stage 13 - Strings, tries and pattern matching

Prerequisites: Stages 2, 7, 10-12.

Objective: Represent shared prefixes and repeated structure precisely.

Core topics: Tries; KMP/prefix function or Z-function; exact matching; rolling-hash collisions; palindromes; bytes versus Unicode.

Recommended study: Handbook: strings. CP-Algorithms: prefix function or Z-function.

Build task: Build a trie and a linear-time matcher. Compare with naive matching on short random strings; test repetition, overlaps and empty patterns where supported.

Practice ladder

- Implement Trie (Prefix Tree) (M).
- Find the Index of the First Occurrence in a String (E, use linear-time matching).
- Word Search II (H).
- Shortest Palindrome (H).

Variation: Move from one pattern to many. Explain trie reuse; explore Aho-Corasick in Stage 17.

Exit gate: Explain fallback/overlap handling, preserve traversal state and bound total input work. Specify alphabet assumptions; hashing needs a collision argument.

Stage 14 - Range queries and ordered information

Prerequisites: Stages 2, 4-6.

Objective: Maintain summaries that support efficient updates and queries.

Core topics: Coordinate compression; Fenwick/segment trees; associative merge and identity; offline sweeps; lazy propagation; order statistics; sparse-table conditions.

Recommended study: Handbook: range queries. CP-Algorithms: Fenwick and segment trees.

Build task: Build Fenwick and segment trees. Compare random updates/queries with a plain array; then add one lazy range-update variant.

Practice ladder

- Static Range Sum Queries (CSES).
- Range Sum Query - Mutable (M).
- Count of Smaller Numbers After Self (H).
- Count of Range Sum (H).

Variation: Switch static data to online updates or sums to minima; reassess the required algebraic properties.

Exit gate: Define boundaries and merge/identity; prove query decomposition. Compression preserves order, not geometric distance.

Stage 15 - Advanced graphs and reductions

Prerequisites: Stages 6-8, 12.

Objective: Turn a difficult graph problem into a known structure or optimization problem.

A. Connectivity and tree queries

Core topics: Disjoint sets, Kruskal, binary lifting and ancestor queries.

Build task: Implement DSU and binary lifting. Check small connectivity and ancestor queries against simple traversal.

Practice ladder

- Redundant Connection (M); Company Queries II (CSES).

B. Graph decomposition

Core topics: Bridges/low-link values; strongly connected components (SCCs); condensation DAGs; Euler trails.

Build task: Implement one SCC algorithm; verify components on small graphs. For undirected bridge code, track edge identity so parallel edges are handled correctly.

Practice ladder

- Critical Connections in a Network (H); Reconstruct Itinerary (H); Coin Collector (CSES).

C. Flow and matching

Core topics: Residual networks; maximum flow; bipartite matching; reductions.

Build task: Implement a residual-flow solver. Check tiny cases by enumeration and verify a flow/cut certificate.

Practice ladder

- Download Speed and School Dance (CSES).

Variation: Model resource assignment as matching. Show both directions: every feasible assignment gives a graph solution, and every graph solution gives a valid assignment.

Exit criteria: For each subunit, justify the invariant, validate an implementation and solve a fresh application. Include both graph decomposition and a flow/matching reduction in the stage assessment.

Recommended study: Erickson: graphs and flows. Handbook: spanning trees, connectivity and flows. Use CP-Algorithms for implementation details after deriving the invariants.

Stage 16 - Counting, number theory and geometry

Prerequisites: Stages 1, 4, 10-12.

Objective: Solve problems whose main difficulty is mathematical structure.

Core topics: GCD; modular arithmetic/inverses; counting; inclusion-exclusion; digit DP; bit operations; cross products; rational slopes.

Recommended study: Handbook: mathematics and geometry. CP-Algorithms: targeted references.

Build task: Implement gcd, modular exponentiation and integer orientation. Check counting formulas by enumeration; normalize slopes and handle duplicate points.

Practice ladder

- Number of Digit One (H).
- Max Points on a Line (H).

Variation: Change a digit/base/range constraint. Extend collinearity to segment intersection.

Exit gate: Derive the counting argument, state inverse conditions and bound intermediate integers. Avoid floating-point equality for exact geometry.

Stage 17 - Composition and the expanding frontier

Prerequisites: Stages 1-16.

Objective: Combine techniques and fill specialist gaps revealed by unfamiliar problems.

Core topics: Meet in the middle; DP optimization conditions; advanced tree/range operations; automata; suffix structures; game DP; computational limits.

Recommended study: USACO Guide, CP-Algorithms and CSES Advanced Techniques, selected by need.

Build task: Combine two techniques, such as a sweep and a range structure. Learn one new specialist technique, state its preconditions and implement it with an independent checker.

Practice ladder

- CSES Advanced Techniques or USACO Guide problems selected from your gaps.

Variation: Solve a fresh task, change one structural assumption and determine whether the method survives.

Exit gate: Solve two new combination problems, complete the specialist implementation and pass the Hard benchmark. Revisit this stage when new gaps appear.

Stage gates and mastery criteria

Complete a stage

1. Complete its build task, core practice ladder, variation and exit criteria.
2. Solve two additional fresh applications independently. Normally allow 60 minutes each at an appropriate introductory/Medium level. Stage 1 may use Easy tasks.
3. In Stages 12-16, include at least one Hard or substantial CSES task, allowing up to 90 minutes. Stage 15 also requires a fresh application from each subunit; Stage 16 must span two subtopics.
4. After at least seven days, pass a closed-notes reconstruction or fresh transfer task. You may start the next compatible stage while this check is pending.

A 90-minute task can span planned study sessions. Count total active time and do not study its solution between sessions. Agree on each block before starting or resuming. Stage 17 adds combination tasks, a specialist implementation, and the Hard benchmark.

Score independent attempts

Dimension	2 - Sound	3 - Strong
Model and approach	Finds a valid approach independently.	Chooses an efficient approach; explains alternatives.
Correctness	Gives a coherent invariant, induction or case argument.	Handles assumptions and difficult boundaries convincingly.
Implementation	Working code with minor self-corrected issues.	Correct code, useful tests, no unresolved defects.
Complexity and limits	Correct time/space bounds and key assumptions.	Accounts for hidden work, expanded state and tradeoffs.

Score 0-1 when the work falls below "Sound." Pass at 10/12 or higher, no dimension below 2, and implementation at 3. Check against reliable tests/solutions; seek outside review when a proof is uncertain.

Assess without AI, editorials, hints or solution templates; compiler feedback and your own tests are allowed. Recently studied or assisted solutions count as learning, not independent mastery.

Milestones

Foundation: reconstruct core structures and solve fresh foundational tasks with valid reasoning.

Independent Medium: solve 8 of 10 fresh mixed Medium problems within 45 minutes each, with at least one success in every sampled family.

Hard development: solve fresh Hard problems with sound reasoning in up to 90 minutes.

Advanced Hard: pass the benchmark in The 45-minute Hard benchmark.

If you miss a gate

Identify the cause: modeling, missing concept, proof, code, complexity or speed. Do one targeted repair session and a related application, then reassess on a different problem after a delay. Repeated failure means return to the prerequisite or reduce difficulty.

Solved-problem totals are not graduation criteria. Keep failed retention skills in the review rotation.

The 45-minute Hard benchmark

Begin after broad topic coverage and reliable Medium performance.

1. Select 20 unseen problems

Have a reviewer or selector choose published Hard problems you have never attempted and whose solution ideas you have not studied. Use two problems from each family; assign each problem one primary family.

Families 1-5	Families 6-10
Arrays / windows	Graph structure / reductions
Search / greedy	Basic / sequence DP
Stacks / heaps	Advanced DP
Trees	Strings
Graph paths / traversal	Range queries / counting / math / geometry

Hide tags, acceptance rates, editorials and technique names added by the selector. Replace accidental prior exposure before starting; keep valid selections even if they look difficult.

2. Run each attempt

Start on first reading. Use Go initially (see Language practice), an editor, compiler, statement and judge. Do not use AI, web search, conceptual hints, editorials or solution templates. Ordinary syntax completion is allowed. Generative completion is forbidden.

All coding, testing, debugging, explanation, and resubmissions count within 45 minutes. Pause only for genuine infrastructure failure. Reserve another 15 minutes for review.

Suggested checkpoints: 0-7 model; 7-15 derive and justify; 15-35 implement; 35-45 test and explain. Adjust the split to the problem.

3. Count a success

A pass requires a correct/accepted solution within 45 minutes, an approach that meets the constraints, and a sound correctness/complexity explanation. Apply the rubric in Stage gates and mastery criteria. A timed-out problem remains a failure in that block even if solved later.

4. Repeat the result

Target 16/20 successes, with at least one success in every family. Repeat on a second fresh block after an interval of at least two weeks; repair weaknesses and maintain older skills between blocks.

One block contains 20 attempts. Repairs add study time. Report the result as performance on this sample, not an 80% guarantee across all Hard problems. Rotate rare topics into later blocks.

Maintain the skill

After an interview cycle, continue weekly practice with a duration chosen before each session. Alternate mixed tasks with core implementation and proof refreshers. Keep timed assessment limits fixed. Before the next cycle, use fresh timed tasks to find gaps.

Language practice: Go, C++ and Rust

Starting point: Go is familiar, C++ is partly familiar, and Rust is new. Algorithm mastery remains the main goal. Language practice fits within the study time chosen for each session. See Session planning.

Choose the exercise language

Use Go for the baseline, first encounters with new algorithm concepts, and initial timed assessments.

Use C++ for selected builds and review exercises. Then solve a fresh related problem in C++. Practice appropriate use of the language and its standard library.

Before Rust algorithm work, learn basic syntax, tests, collections and ownership rules for memory management. Study borrowing, which allows access without transferring ownership. Then implement small algorithms that you already understand.

Practice without tripling the course

Choose one language per exercise. Revisit selected algorithms in another language after a delay. Close the original implementation and reconstruct the solution. Use the same expected results to test each implementation.

Use selected build and review slots for language practice. Keep the required fresh applications and delayed stage checks. Expect a longer course when study time also develops language fluency.

Assess and record

Before timed assessments in C++ or Rust, independently solve and test fresh small problems in that language. Record the evidence. Choose the assessment language before seeing the task.

Record the language for every attempt. Track algorithm reasoning and language errors separately. Keep the correctness criteria and time limits unchanged. Language errors still count against implementation and timing.

Reimplementing a familiar problem is practice. It does not count as a fresh-problem assessment. Report assessment results with the language used.

Libraries and references

After building a heap, use Go container/heap, C++ std::priority_queue, or Rust BinaryHeap for later exercises. Read the reference for the selected language as needed. The Rust Book provides the introductory sequence.

The Rust Book Go heap C++ priority queue Rust heap

Session planning

Tell the tutor your available active minutes before each session. If you omit the duration, the tutor asks before starting. If your window includes planned breaks, subtract them first. Planning and setup do not start the timer.

Calculate the ordinary study split

Retrieval means recalling prior learning without notes. Reserve one-sixth each for retrieval and feedback. Round each to whole minutes, with halves rounded up and a minimum of one minute. Assign the rest to focused work. Use at least three active minutes.

Available	Retrieval	Focused work	Feedback
20 min	3 min	14 min	3 min
30 min	5 min	20 min	5 min
45 min	8 min	29 min	8 min
60 min	10 min	40 min	10 min
90 min	15 min	60 min	15 min
120 min	20 min	80 min	20 min

Choose work that fits

Retrieval: select due reviews that fit, starting with weak skills. If none are due, use a baseline prompt.

Focused work: follow the weekday theme, prerequisites, and unfinished work. With 20-30 minutes total, target one model, trace, proof step, or repair. With 45-60 minutes, attempt a small exercise or a larger task segment. With 90-120 minutes, plan smaller work blocks, with a related application if progress permits.

Feedback: review the first unresolved gap, save the work, and record the exact next task. Protect this time when a task needs another session.

Before study, the tutor states the purpose, budgets, independent output, success criteria, and stopping point. The plan separates professor demonstrations from tutor practice. Support remains available without revealing claims or strategies before an attempt.

If the result is achieved early, move to a useful application or close the block. After two unsuccessful prompts, offer help without revealing it. Wait for the learner to choose. Protect feedback time and carry unfinished work forward.

At close, record the whole attempt, assistance, first substantive gap, learner feedback, and next adjustment. Keep fresh baseline attempts in the plan. Planning and maintenance are not evidence of learning.

Keep assessment standards fixed

Reserve 60 minutes for a 45-minute benchmark plus review, or 75 minutes for a 60-minute stage task plus review. A 90-minute task needs 105 active minutes including review and can span agreed study blocks. Give no feedback or solution study before the attempt ends. If an uninterrupted assessment and review do not fit, choose ordinary study or schedule the assessment later. Extra time can fund separate work after review. It never extends the assessment deadline.

These splits are planning rules, not measured optimal learning intervals. The research and its limits remain in Selected references. The timer tracks active time and reported pauses. It does not send deadline alerts.

Recommended resource stack

Choose one explanation source for the current topic. Read only what supports the next exercise; use topic names because book chapter numbers vary by edition.

Core study and practice

MIT 6.006. Foundation curriculum. Use notes, assignments / Problem Set 0, practice problems and quizzes. Videos are optional. Use Go first, then the selected exercise language.

The Algorithm Design Manual. Practical explanations and an algorithm reference catalog. Use your existing copy; verify community-contributed solutions.

LeetCode. Use the linked stage ladders for practice and separate unseen problems for assessments. Platform difficulty labels are approximate.

CSES Problem Set. Broader implementation practice, especially advanced topics. If a judge lacks the exercise language, use local tests with independently established expected results.

Targeted depth

Algorithms - Jeff Erickson. Design, proofs, DP, graph algorithms and flows.

Competitive Programmer's Handbook. The "Handbook" cited in stages: concise techniques for graphs, strings, mathematics and range queries. Use examples in the exercise language.

CP-Algorithms. Reference for a specific technique or implementation detail.

USACO Guide. Optional advanced explanations and problem selection. Use the modules that address a demonstrated gap.

Selected references

Learning research

The course applies findings from retrieval practice, spacing, worked examples and mixed practice. Evidence comes from multiple learning domains, not a trial of this algorithms course. Review intervals, time limits and score thresholds are adjustable course-design choices.

Dunlosky et al. (2013) - review of learning techniques, including practice testing and distributed practice.

Karpicke & Blunt (2011) - retrieval and meaningful learning in science-text tasks.

van Gog & Rummel (2010) - learning from examples and adapting guidance.

Rohrer et al. (2020) - interleaved mathematics practice.

Karpicke & Bauernschmidt (2011) - spaced retrieval and the limits of prescribing one schedule.

IES practice guide (2007) - spacing, worked examples with problem-solving, retrieval, and explanatory questions.

Updated 28 October 2026 for tutor and professor roles, requested help, and independent attempts. Practice links and assessment standards remain unchanged. Problem statements and solutions remain with their original publishers.